

Competitive Exam Practice 2 — Answer Key

Number Systems, Representation, and Arithmetic Hardware

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

August 10, 2026

Q1 (C). $43 = 32+8+2+1 = 00101011$ in 8-bit binary. Since 43 is non-negative, its 2's complement representation is the plain binary value. (01010101 = 85; 11010101 and 10101011 have the sign bit set, so they are negative.)

Q2 $-2^{n-1} \leq N \leq 2^{n-1} - 1$. With n bits in two's complement, the MSB is the sign bit; the most negative value has MSB 1 and all else 0 (-2^{n-1}), and the most positive has MSB 0 and all else 1 ($2^{n-1} - 1$). One of the 2^n bit patterns is wasted by reserving it for the single representation of zero.

Q3 (C). Squaring both sides: $(224)_r = (13)_r^2$. Expanding: $2r^2 + 2r + 4 = (r + 3)^2 = r^2 + 6r + 9$. Rearranging: $r^2 - 4r - 5 = 0$, i.e. $(r - 5)(r + 1) = 0$. Since $r > 4$ (digit 4 appears), $r = 5$.

Q4 15 full adders, 1 half adder. A ripple-carry adder adds bit 0 with no carry-in, so it needs only a half adder (or a full adder with $C_{in} = 0$); every other bit position (1 through 15) needs a full adder to accept the carry from the previous stage. Hence $n - 1 = 15$ full adders and 1 half adder.

Q5 (C). $539_{10} = 0x21B$. The 2's complement of a positive number in k bits is $2^k - x$. Using 12 bits: $-539 = 4096 - 539 = 3557_{10} = 0xDE5$. (Option (C) as printed in the book is DE5; the OCR reading "DES" is a 5/S glyph ambiguity.)

Q6 (B). Critical path through the carry-lookahead adder:

- **Generate propagate/generate:** $P_i = A_i \oplus B_i$ and $G_i = A_i \cdot B_i$. XOR is two gate levels (two-level AND-OR), so delay = 2.
- **Carry network:** each C_i is a two-level AND-OR expression (sum of products), delay = 2.
- **Sum:** $S_i = P_i \oplus C_i$, another two gate levels, delay = 2.

Total = $2 + 2 + 2 = 6$ time units.

Q7 (A). Booth's algorithm recodes the multiplier by examining adjacent bit pairs (Q_0, Q_{-1}) : it adds/subtracts only at a $0 \rightarrow 1$ or $1 \rightarrow 0$ transition and does nothing on a run of identical bits (00 or 11). A strictly alternating pattern $101010 \dots 1010$ has a transition at *every* bit position, forcing an add or subtract on every single cycle — the maximum possible number of arithmetic operations. Runs of identical bits (options B–D) let Booth's algorithm collapse a whole run into a single add/subtract at its boundary, which is exactly the speedup the algorithm is designed to exploit.

Q8 8. The restoring division algorithm (as in the lecture: A = remainder, Q = dividend $\rightarrow$ quotient, M = divisor) repeats its shift-subtract-restore-or-set-bit cycle exactly n times to produce an n -bit quotient. For two 8-bit operands, $n = 8$, so the loop executes 8 cycles, each producing exactly one quotient bit Q_0 — regardless of how many of those cycles also perform a restoring add.